

Ensamblador ARM

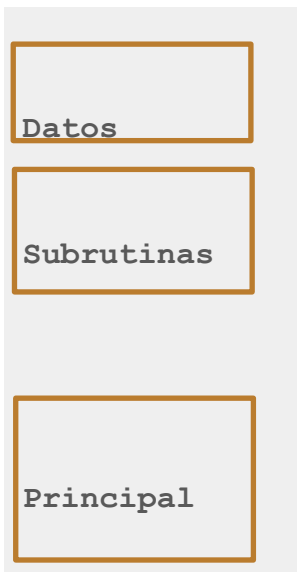
Llamada a procedimientos. Subrutinas
Ejemplos ARM

Mapeo de estructuras de control de flujo

Instrucciones de control de flujo: Llamada a Procedimientos

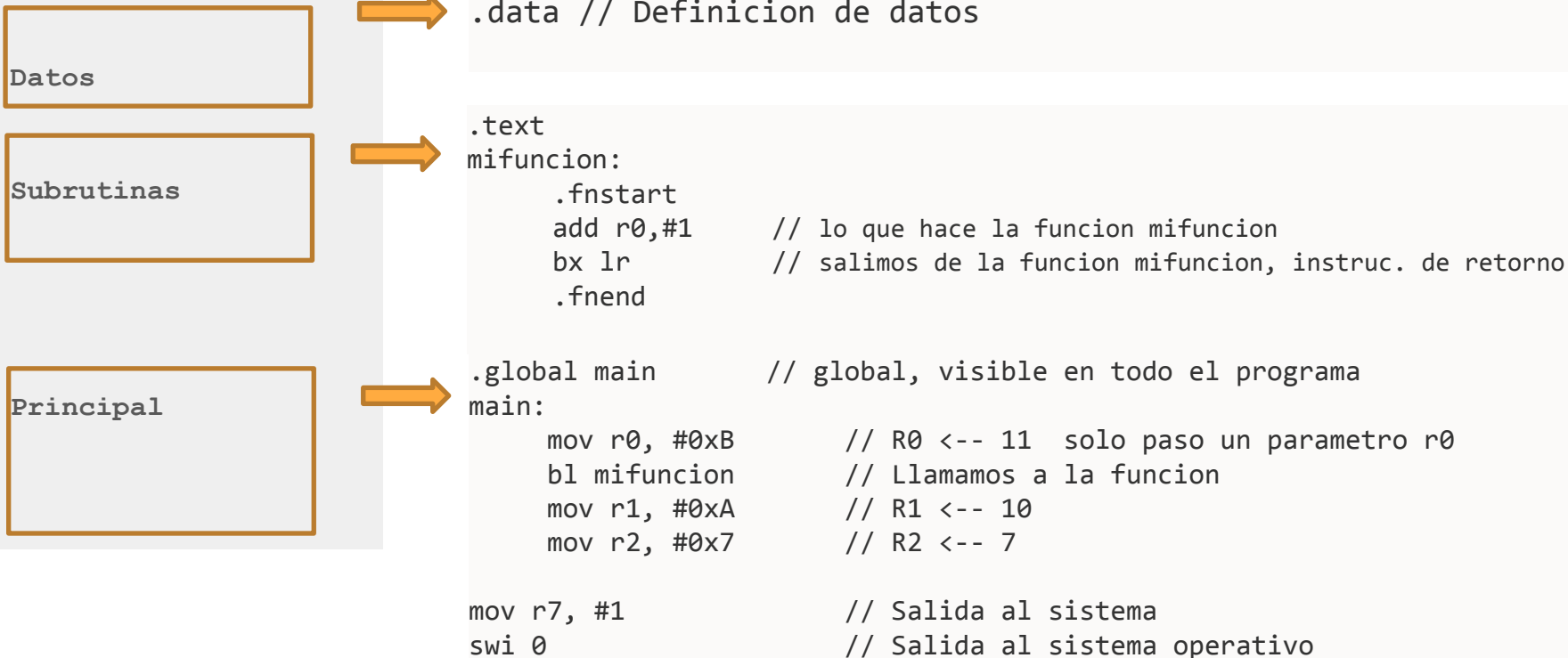
Bibliografía

Organización de computadoras: un enfoque estructurado.
Andrew Tanenbaum. Capítulo 5



Subrutinas en ensamblador llamadas desde ensamblador

En este ejemplo crearemos nuestras propias funciones con pasaje de parámetros o argumentos



Subrutinas:

```
.data
```

```
.text
```

```
mifuncion:
```

```
    .fnstart
```

```
        add r0,#1                // lo que hace la función mifuncion
```

```
        bx lr                    // salimos de la función mifuncion
```

```
    .fnend
```

```
@ ----- Código del main -----
```

```
.global main                // global, visible en todo el programa
```

```
main:
```

```
    mov r0, #0xB              // R0 <-- 11. Solo paso un parámetro r0
```

```
    bl mifuncion              // Llamamos a la función
```

```
    mov r1, #0xA              // R1 <-- 10
```

```
mov r7, #1                    // Salida al sistema
```

```
swi 0                        // Salida al sistema operativo
```

Subrutinas anidadas

En el ejemplo anterior vimos un sencillo esquema que vale para un sólo nivel de subrutinas, es decir, dentro de subrut no podemos llamar a otra subrutina porque sobrescribimos el valor del registro **lr**.

La solución para extender a cualquier número de niveles es almacenar el registro **lr** en pila con las instrucciones **push** y **pop**

```
main :
    mov r1, #1
    mov r2, #2
    bl nivel1 // No es necesario guardar lr
    mov r5, #5 // Siguiete instrucción
    ...
    /* ---- Subrutinas ---- */
nivel1:
    push {lr}          // como esta funcion llama a otra
    mov r3, #3          // se guarda lr, porque sino se pierde
    bl nivel2          // llama a la funcion anidada
    pop {lr} // restauramos el lr de esta funcion bx lr
    /* ----- */
nivel2:
    mov r4, #4          // no es necesario guardar lr pq
    bx lr              // es la última función
```

Subrutinas: Convención AAPCS

Podemos seguir nuestras propias reglas, pero si queremos interactuar con las librerías del sistema, tanto para llamar a funciones como para crear nuestras propias funciones y que éstas sean invocadas desde un lenguaje de alto nivel, tenemos que seguir una serie de pautas, lo que se denominamos **AAPCS (Procedure Call Standard for the ARM Architecture)**

Subrutinas: Convención AAPCS

- Podemos usar hasta cuatro registros (desde r0 hasta r3) para pasar parámetros y hasta dos (r0 y r1) para devolver el resultado.
- Si por ejemplo la función sólo usa dos parámetros de tipo int con r0 y r1 nos basta. Lo mismo pasa con el resultado, podemos no devolver nada (tipo void), devolver sólo r0 (tipo int ó un puntero a una estructura más compleja), o bien devolver r1:r0 cuando necesitemos enteros de 64 bits (tipo long long).

R0	Registros Generales
R1	
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	Registros Especiales
SP (R13)	
LR (R14)	
PC (R15)	

Buenas prácticas

Usando estos criterios podemos organizar mejor el código en un lenguaje que no tiene estructura.

- Tener en cuenta que las etiquetas no se pueden repetir.

Otras buenas prácticas:

- Implementar funciones con un único punto de entrada y de salida
- Respetar la convención de llamadas a función
- Apilar y desapilar registros que hay que preservar
- ¿Qué otros criterios podríamos agregar?